Design a Chat Application (aka WhatsApp)

Mastering System Design: From Basics to Cracking Interviews



Send Message

Receive Message

API Gateway
Auth, Rate Limiting

Chat Service
Message Processing

User Service
Contacts, Presence

Notification Service
Push, Email, In-App

Message Broker
Kafka / SQS

Database
Messages, Users
Conversations

Object Storage
Media Files
(Images, Videos, Docs)

Real-time Workers
Message Delivery
Presence Updates

Notification Workers
Push Notifications
Email / SMS

Instructor: Rahul Rajat Singh

What are we building – Functional Requirements?

We're designing a **real-time chat application like WhatsApp.**

Functional Requirements

1. **1-to-1 and group messaging**
   Send and receive messages in individual and group chats.
2. **Typing indicators and online status**
   Show typing status and whether users are online.
3. **Media support (images, videos, documents)**
   Share photos, videos, and files.
4. **Delivery and read receipts**
   Show message delivery and read status.
5. **Multi-device support with sync**
   Access messages across multiple devices in sync.



Non-Functional Requirements

Sub-second Message Delivery

Messages should be delivered in sub-second time.

High Availability & Fault Tolerance

System should be highly available and resillient to failures.

End-to-End Encryption

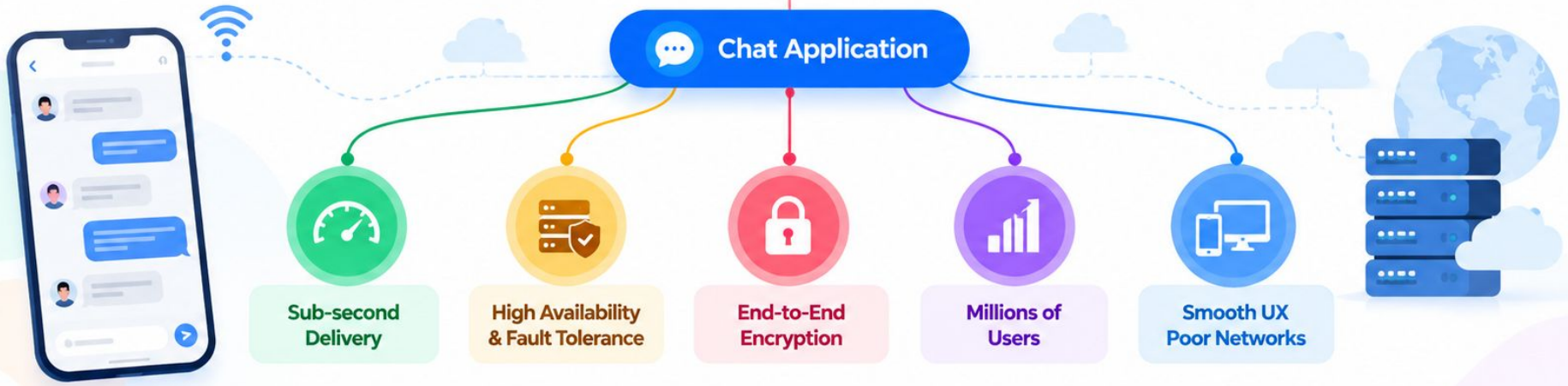
All messages should be encrypted end-to-end for privacy and security.

Scalable to Millions of Users

Should handle millions of concurrent users.

Smooth UX Even on Poor Networks

Provide a seamless user experience even on low bandwidth or unreliable networks.



Constraints & Challenges



1 Message Ordering & Delivery Guarantees

Ensure no duplicates, no lost messages, and correct order, even under retries or reconnections.



2 Unreliable Networks

Mobile users may frequently disconnect; we need robust retry, buffering, and sync mechanisms.



3 Security & Privacy

End-to-end encryption must be seamless, with no access to message content by servers.



4 Latency Expectations

Sub-second message delivery is expected, regardless of load or geography.



5 Presence & Sync

Track who's online and sync message state across all devices in near real-time.



6 High Scale

Millions of concurrent users, chatrooms, and messages per second; we must scale horizontally and efficiently.



Estimating Scale

We begin with a few rough assumptions to get a sense of expected load:

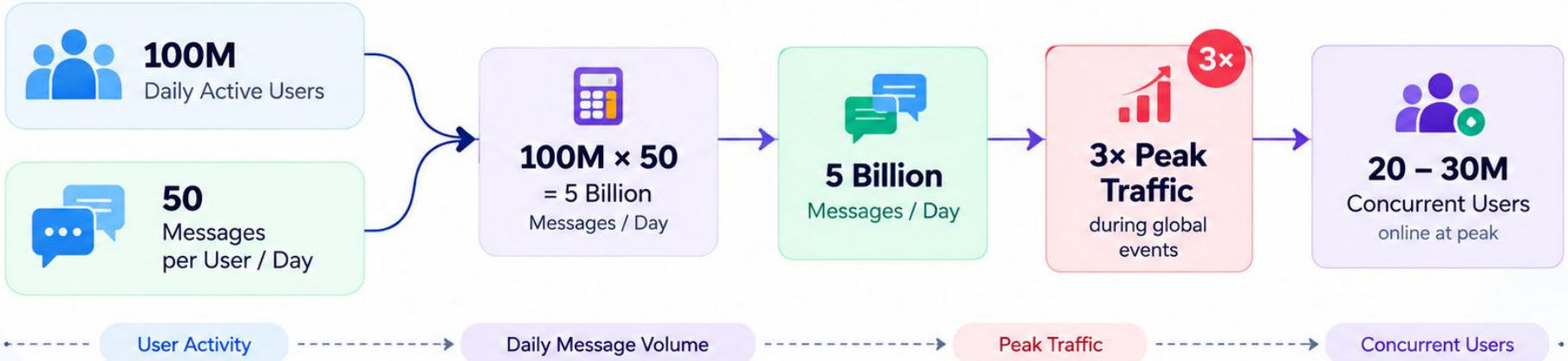
100 Million
Active users / day

50
Average messages per user / day

5 Billion
Messages / day
(100M × 50)

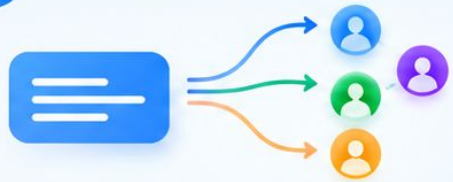
3×
Peak traffic multiplier
during global events

20 – 30 Million
Concurrent users online at peak



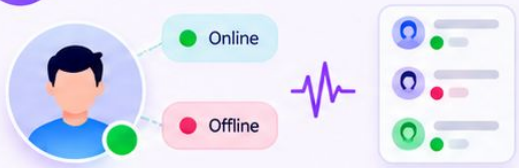
Identifying System Bottlenecks and Challenges

1 Message Ingestion & Fanout



- Fanout to multiple recipients/devices **multiplies delivery load**.
- Queue, buffer and batch intelligently (Kafka, Redis Streams).

2 Presence Updates



- Real-time online/offline tracking is **high-frequency**.
- Use **Pub/Sub** and efficient **TTL-based caching**.

3 Delivery Acknowledgments



- Sent/read receipts add writes and can **overload databases**.
- Use fast, write-optimized storage (Cassandra or DynamoDB).

4 Sync Across Devices



- Messages must **sync across all logged-in devices.**
- Use a **device registry** and smart **deduplication.**

5 Storage & Retrieval



- Store **billions of searchable messages.**
- Partition carefully and use **append-friendly stores.**

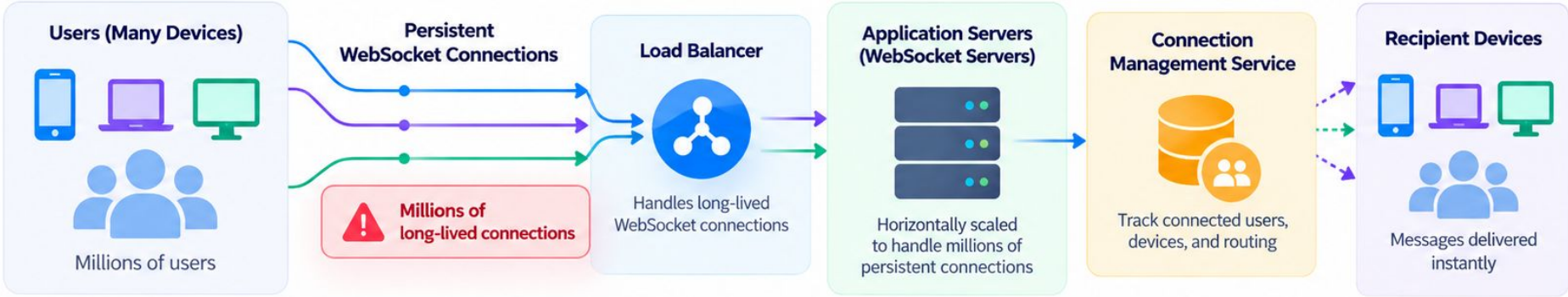
6 Encryption & Security



- End-to-end encryption increases **CPU cost and complexity.**
- Key exchange and **secure metadata handling** must scale too.

Key Bottleneck: Real-Time Delivery

WebSockets



Persistent Connections

Maintain open connections for instant message delivery using WebSockets.

Scale Infrastructure

Load balancers and application servers must support long-lived sessions at large scale.

Connection Registry

Track connected users, devices, and message routing.

Unstable Networks

Handle connection churn, network drops, and mobile limitations (e.g., background app states).

Why it's a bottleneck: Handling millions of concurrent WebSocket connections requires optimized infrastructure, horizontal scaling, and careful connection lifecycle management.

High-Level Architecture Overview & Key Components

1 Connection Manager (WebSocket Service)
- Manages persistent WebSocket connections
- Delivers real-time messages
- Tracks user/device sessions

2 Chat Service
- Core business logic for sending and receiving messages
- Stores messages
- Manages delivery status and history

3 Presence Service
- Tracks online/offline status
- Typing indicators
- Synchronizes across devices using pub-sub or Redis

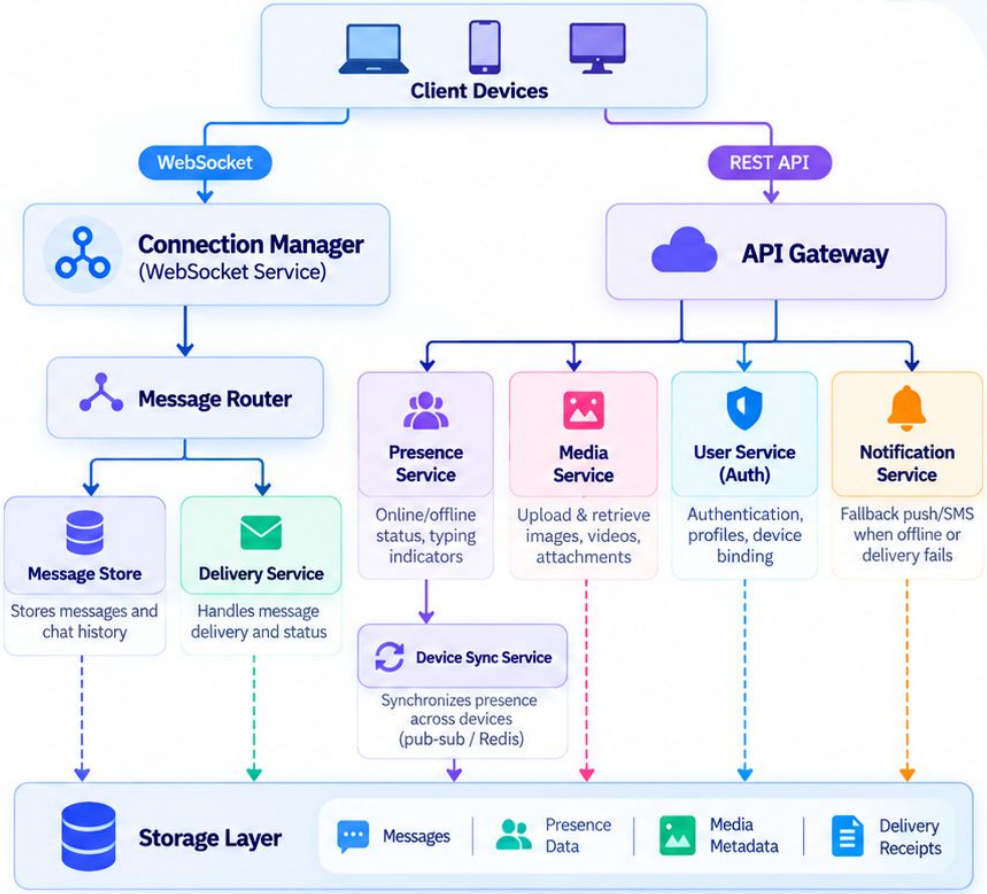
4 Notification Service
- Handles fallback notifications (e.g., push, SMS)
- When recipients are offline or WebSocket delivery fails

5 Media Service
- Manages upload and retrieval of media files (images, videos, attachments)
- Uses object storage

6 Auth & User Service
- Authentication
- User profile management
- Device binding

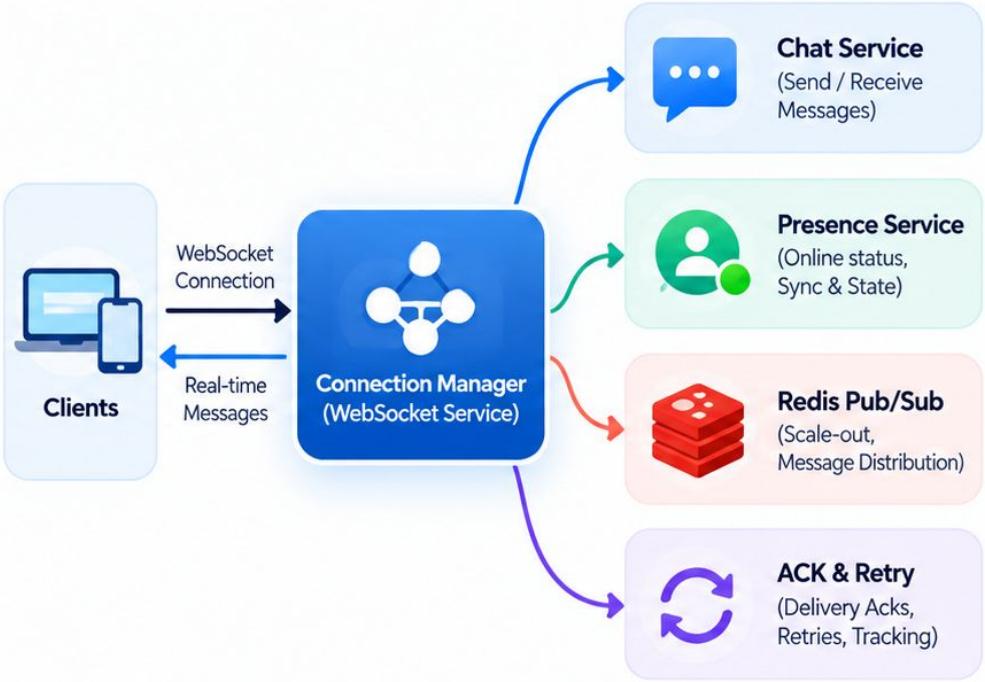
7 Storage Layer
- Stores messages, presence, media metadata, and delivery receipts
- Optimized for high write throughput

8 Group Service
- Group creation, deletion
- Membership management (add/remove users)



Real-Time Connection Manager (WebSocket Service)

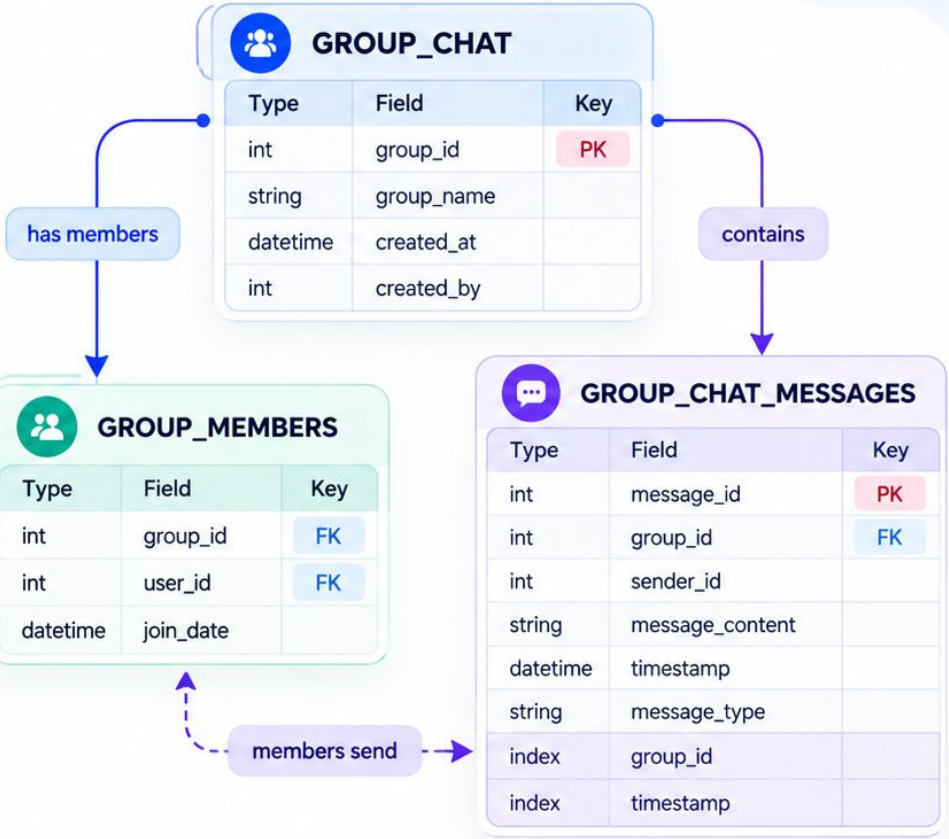
1. Maintains **long-lived WebSocket** connections per user and device.
2. **Routes messages** between senders and recipients.
3. Integrates with **Presence and Chat services** for sync and state management.
4. **Scales out** using sticky sessions, Redis Pub/Sub, or message queues.
5. Supports **delivery acknowledgments**, retries, and device-level delivery tracking.



DB Schema for Direct Chat

DIRECT_CHAT_MESSAGES

Type	Field	Key
int	message_id	PK
int	sender_id	
int	receiver_id	
string	message_content	
datetime	timestamp	
string	status	
string	message_type	
index	sender_id	
index	receiver_id	
index	timestamp	



How Chat & Group Chat Services Use Connection Manager for Real-Time Messaging

Direct Chat

Send a message to a single user

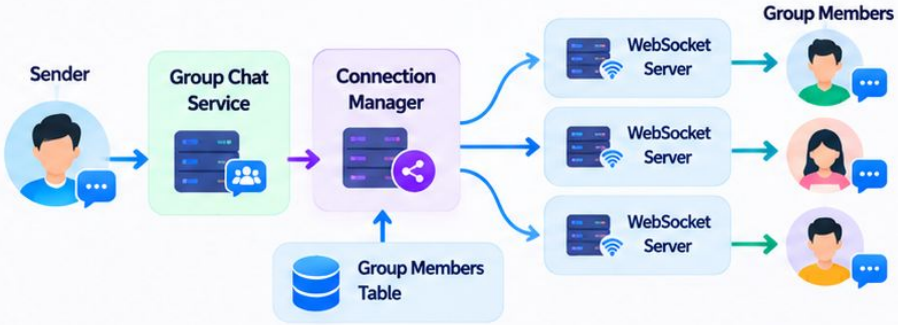
1. Sender sends a message to the **Chat Service** with the recipient's user ID.
2. The Chat Service then sends the message to the **Connection Manager**.
3. Connection Manager looks up the recipient's active WebSocket connection and forwards the message to the corresponding **WebSocket server**.
4. The Receiver receives the message in real-time.



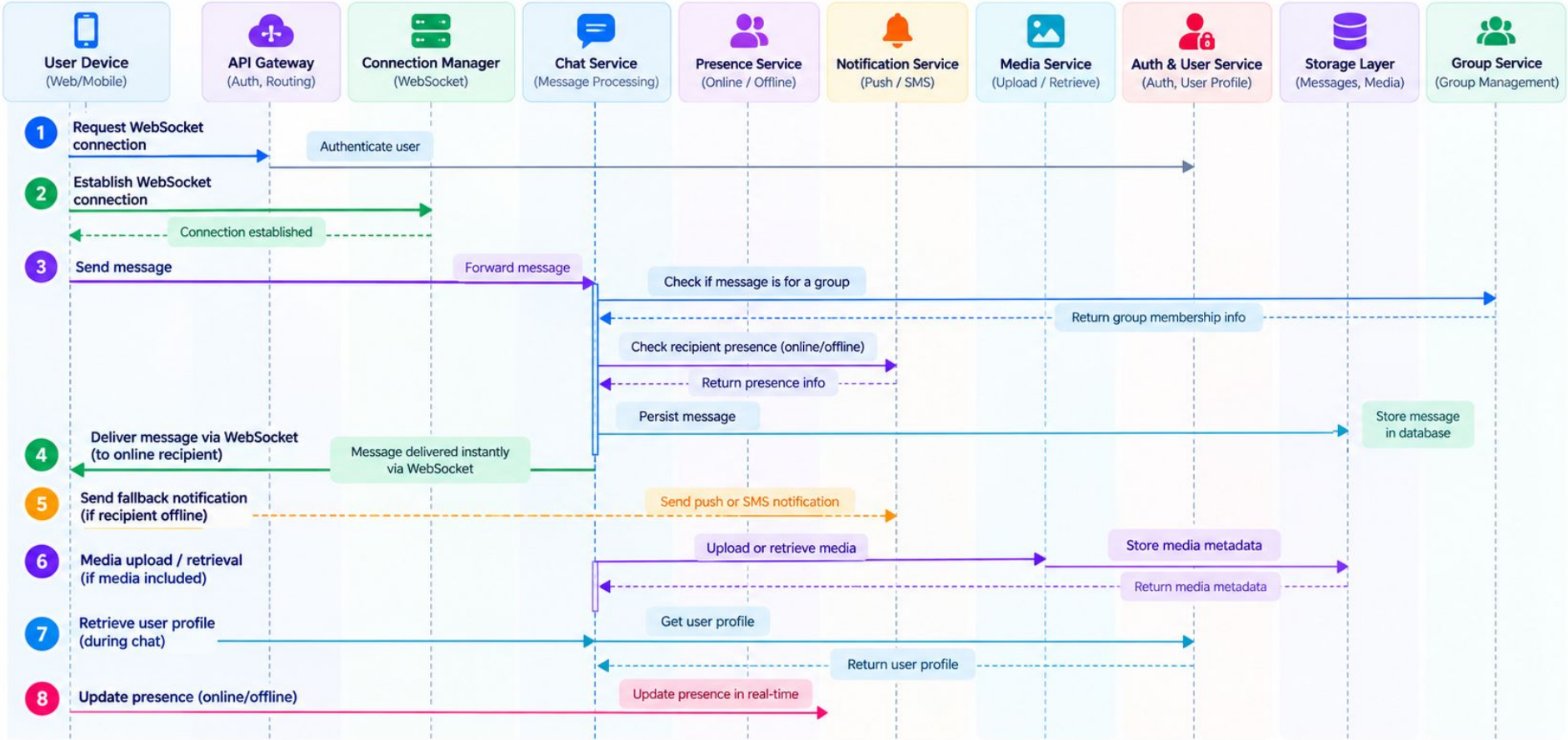
Group Chat

Send a message to multiple users in a group

1. Sender sends a message to the **Group Chat Service** with the group ID and message content.
2. The Group Chat Service queries the **Connection Manager**, which looks up all active WebSocket connections for users in that group via the Group Members table.
3. The Connection Manager forwards the message to all group members' **active WebSocket connections**.
4. All group members receive the message simultaneously.



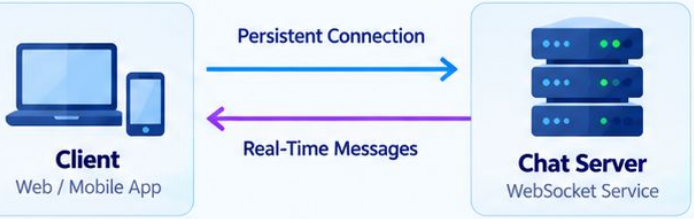
Chat Application — Message Flow & Sequence Diagram



API Design — WebSocket + REST

WebSocket • Real-Time

Persistent, bidirectional connection for real-time messaging



Key WebSocket Events

connect	Initiates persistent connection with access token and user metadata.
send_message	JSON payload with recipient ID, message content, timestamp, type.
message_ack	Acknowledgment from client upon delivery/read.
typing_indicator	Optional message indicating typing state.

REST APIs • On-Demand

For non-real-time needs like history, uploads, presence and feedback

GET `/messages?userId=&conversationId=`

Fetch historical messages with pagination and filters.

POST `/media/upload`

Upload image/video/audio attachments and get media URL.

GET `/presence/{userId}`

Fetch last seen / online status for a user.

POST `/feedback`

Submit crash reports or feedback (optional).

Security by Default

Secure and role-based access for all endpoints.



JWT Authentication

All requests (WebSocket handshake + REST) require JWT authentication.

Role-based Access

Role-based access for user vs. admin endpoints.

Strategic Tech & Infrastructure Decisions



TECH STACK

Message Broker
Kafka (high-throughput) or AWS SQS (managed service)

Real-time Communication
WebSockets for instant messaging, leveraging SignalR (for .NET) or similar frameworks

Push Notifications
Firebase Cloud Messaging (FCM) for push notifications

Database
NoSQL databases (e.g., MongoDB) for fast reads and flexible schema

INFRASTRUCTURE & DEPLOYMENT

Deployment
Microservices on Kubernetes for scalability and management

Scaling
Horizontal scaling for WebSocket connections; KEDA (Kubernetes Event-Driven Autoscaling) for auto-scaling workers

Storage
Redis for caching active user sessions, PostgreSQL for transactional data (e.g., user settings)

SECURITY & OBSERVABILITY

Authentication
OAuth 2.0 for secure authentication across platforms

Logging & Monitoring
Prometheus and Grafana for real-time monitoring; CloudWatch for logs and alerts

High Availability
Multi-region deployment for failover; load balancing across WebSocket servers

The Final Design - Chat Application

